1. Quick Sort

Quick sort is a divide and conquer based sorting algorithm. It works by selecting a pivot element and partitioning the array such that:

- elements smaller than the pivot are placed on the left
- elements greater than the pivot are placed on the right.

The same process is recursively applied to the sub arrays.

⇒ Algorithm (Quick Sort)

QUICK-SORT (A, low, high)

1. if low < high then:
2. Find the pivot index p using PARTITION (A, low, high)
3. Recursively apply Quick Sort on left subarray:
   QUICK-SORT (A, low, p-1)
4. Recursively apply Quick Sort on right subarray:
   QUICK-SORT (A, p+1, high)

PARTITION (A, low, high)

1. Set pivot = A[high]
2. Set i = low - 1
3. For j = low to high - 1:
   If A[j] ≤ pivot:
   increment i
   swap A[i] and A[j]
4. Swap A[i+1] and A[high]
5. Return i+1 (pivot position)

⇒ Example

Input array: [10, 7, 8, 9, 1, 5]

Step 1:

Pivot = 5

After partition – [1, 5, 8, 9, 10, 7]

Step 2:

Left subarray: [1] - already sorted

Right subarray: [8, 9, 10, 7]

Step 3:

Pivot = 7

After partition - [7, 8, 9, 10]

Final Sorted Array:

[1, 5, 7, 8, 9, 10]

⇒ Analysis

1. Time complexity

Best case	$O(n \log n)$	Pivot divides array into two nearly equal halves
Average case	$O(n.\log n)$	Random pivot selection gives balanced partitions
Worst case	$O(n^2)$	Pivot is always smallest or largest element (already sorted array)

2. Space complexity

- $O(\log n)$ - recursive stack in best/average case
- $O(n)$ - worst case recursion

⇒ Advantages

- Faster in practice than many other $O(n.\log n)$ algorithms
- In place sorting (no extra array needed)

⇒ Disadvantages

- worst-case performance is poor
- Not stable

2. Merge Sort

Merge sort is a divide and conquer sorting algorithm that divides the array into halves, sorts them recursively, and then merges the sorted halves.

It guarantees $O(n.\log n)$ time complexity in all cases.

⇒ Algorithm

MERGE-SORT (A, low, high)

1. If low < high:
2. Find mid point:

   mid = (low + high)/2
3. Recursively sort left half

   MERGE-SORT (A, low, mid)
4. Recursively sort right half

   MERGE-SORT (A, mid+1, high)
5. Merge the two sorted halves using MERGE()

MERGE (A, low, mid, high)

1. Create two temporary arrays:

   left array : A[low...mid]

   right array: A[mid+1...high]
2. Compare elements from both arrays
3. Insert smaller element into original array
4. Copy remaining elements

⇒ Example

Input array: [38, 27, 43, 3, 9, 82, 10]

Step 1: Divide

[38, 27, 43, 3] [9, 82, 10]

Step 2: Further divide

[38, 27] [43, 3] [9] [82, 10]

Step 3: Sort and Merge

[27, 38] [3, 43] [9] [10, 82]

Step 4: Final Merge

[3, 9, 10, 27, 38, 43, 82]

⇒ Analysis

1. Time complexity

Best case	$O(n.\log n)$	Always divides into two halves
Average case	$O(n.\log n)$	Consistent divide and merge
Worst case	$O(n.\log n)$	Guaranteed performance

2. Space Complexity

$O(n)$ – requires additional temporary arrays

⇒ Advantages

- Stable sorting algorithm
- Guaranteed $O(n.\log n)$ time
- Suitable for large data sets and linked lists

⇒ Disadvantages

- Requires extra memory
- Slower than quick sort in practice for small arrays.